SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	B.Lakshmi Lohitha	**Reg. No.:**	192324182
Section / Batch:	AI-DS(2023)	**Date:**	27-10-26

Code of Conduct

I B.Lakshmi Lohitha(192324182)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address
Password
Mobile Number

using Regular Expressions.

Validation Rules

Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'.
Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters
At least one uppercase letter
At least one lowercase letter
At least one digit
At least one special character (@,#,$,%,&,!)

Mobile Number

Must contain exactly 10 digits.
First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

ANSWER – Section 1

A. Algorithm / Approach

- Compile one regular expression per credential with the **re** module and test with **fullmatch()**, so the whole string must satisfy the rule.
- **Email:** `^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(?:com|org|edu|net|in)$` – first character a letter, local part letters/digits/dot/underscore, domain alphabets only, extension limited to the five allowed values.
- **Password:** four look-aheads `(?=.*[A-Z])(?=.*[a-z])(?=.*[0-9])(?=.*[@#$%&!])` check each rule independently (any order) and `.{8,}` enforces minimum length 8.
- **Mobile:** `^[6-9][0-9]{9}$` – first digit 6–9 followed by exactly nine digits.
- One function per validator returns the required message; `run_tests()` displays input and output for valid and invalid cases.
- **Complexity:** O(n) time per validation for input length n, O(1) extra space.

B. Source Code – q1_validator.py

```
"""Q1 - Intelligent Email, Password and Mobile Number Validat
or (Regex)."""import re

# Email: starts with a letter; letters/digits/'.'/'_'
before '@';# domain = alphabets only; extension must
be com/org/edu/net/in
EMAIL_PATTERN = re.compile(r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(?:com|org|edu|net|in)$")

# Password: lookaheads enforce each rule, {8,} enforces the length
PASSWORD_PATTERN = re.compile(r"^(?=.*[A-Z])(?=.*[a-z])(?=.*[0-9])(?=.*[@#$%&!]).{8,}$")

# Mobile: first digit 6-9 followed by exactly 9
more digits MOBILE_PATTERN = re.compile(r"^[6
-9][0-9]{9}$")


def validate_email(email: str) -> str:
```

```
    return "Valid Email" if EMAIL_PATTERN.fullmatch(email) else "Invalid Email"
def validate_password(password: str) -> str:
    return "Strong Password" if PASSWORD_PATTERN.fullmatch(password) else "Weak Password"

def validate_mobile(mobile: str) -> str:
    return "Valid Mobile Number" if MOBILE_PATTERN.fullmatch(mobile) else "Invalid Mobile Number"

def run_tests() -> None:
    emails = ["john.doe_1@gmail.com", "1john@gmail.com", "ali
             ce@univ.edu", "bob@site.xyz", "carl@mail123.com", "dave
             @@mail.com", "eve@college.in"]
    passwords = ["Secure@123", "password", "PASSWORD@1", "Pass@
                1", "Abcdefg1","Hello#2026", "NoDigit@abc"]
    mobiles = ["9876543210", "5876543210", "98765", "98765432101", "98765abc10", "6000000000"]

    print("=" * 55); print("EMAIL VALIDATION"); p
    rint("=" * 55)for e in emails:
        print(f"Input: {e:<24} -> {validate_email(e)}")
    print("\n" + "=" * 55); print("PASSWORD VALIDATION");
    print("=" * 55)for p in passwords:
        print(f"Input: {p:<24} -> {validate_password(p)}")
    print("\n" + "=" * 55); print("MOBILE NUMBER VALIDATION"); print("=" * 55)
    for m in mobiles:
        print(f"Input: {m:<24} -> {validate_mobile(m)}")

def main() -> None:
    print("1. Run built-in test cases\n2. Enter my own cr
    edentials")choice = input("Choice: ").strip()
    if choice == "
        1":run_te
        sts()
    else:
        print(validate_email(input("Enter Email    : "
        ).strip())) print(validate_password(input("Enter
        Password : ").strip()))print(validate_mobile(in
        put("Enter Mobile                  : ").strip()
        ))


if __name__ == "__mai
    n__": main()
```

C. Sample Output

Run 1 – built-in valid and invalid test cases:

```
1. Run built-in test cases
2. Enter my own credent
ials Choice: 1
============================================
=========== EMAIL VALIDATION
============================================================
Input: john.doe_1@gmail.com -> Valid
Email Input: 1john@gmail.com       -
> Invalid Email
Input: alice@univ.edu      -> Valid Email
Input: bob@site.xyz        -> Invalid Em
ail Input: carl@mail123.com   -> Invali
d Email Input: dave@@mail.com      -
> Invalid Email
Input: eve@college.in      -> Valid Email

============================================
=========== PASSWORD VALIDATION
============================================================
Input: Secure@123          -> Strong Password
Input: password            -> Weak Password
Input: PASSWORD@1          -> Weak Password
Input: Pass@1              -> Weak Password
Input: Abcdefg1            -> Weak Password
Input: Hello#2026          -> Strong Password
Input: NoDigit@abc         -> Weak Password
```

```
==========================================
========== MOBILE NUMBER VALIDATION
==========================================================
Input: 9876543210          -> Valid Mobile Number
Input: 5876543210          -> Invalid Mobile Number
Input: 98765               -> Invalid Mobile Number
Input: 98765432101         -> Invalid Mobile Number
Input: 98765abc10          -> Invalid Mobile Number
Input: 6000000000          -> Valid Mobile Number
```

Run 2 – user-entered credentials:

```
1. Run built-in test cases
2. Enter my own credent
ials Choice: 2
Enter Email : priya.s_21@saveetha
.edu Valid Email
Enter Password : Nlp@
2026 Strong Password
Enter Mobile: 91234567
80 Valid Mobile Number
```

Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
  - States
  - Input alphabet
  - Transition table
  - Initial state
  - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:
Strings ending with "ab"

Input

abaab

Output

Transition Path:
q0 → q1 → q0 → q1 → q2
Accepted

ANSWER – Section 2

A. Algorithm / Approach

- Formal model: M = (Q, Σ, δ, q0, F). The transition table δ is stored as a dictionary `{(state, symbol): next_state}` giving O(1) lookup.
- **Validation:** the constructor checks that the initial and final states belong to Q and that δ is complete (every state × symbol has one valid target); otherwise a `ValueError` is raised and reported.
- **Simulation:** start at q0; for each input symbol look up δ(current, symbol) and append the new state to the path; a symbol outside Σ rejects the string immediately. After the last symbol the string is **Accepted** if the current state is in F.
- DFA for "strings ending with ab": q0 = no progress, q1 = last symbol was 'a', q2 (final) = last two symbols were 'ab'.
- **Complexity:** O(n) time per string (one step per symbol), O(|Q|·|Σ|) space for the table and O(n) for the stored path.

δ	a	b
→ q0	q1	q0
q1	q1	q2
* q2 (final)	q1	q0

Note on the example: abaab has 5 symbols, so its path contains 6 states. The simulator gives q0 → q1 → q2 → q1 → q1 → q2 (Accepted); the shorter

path in the question paper is only an abbreviated illustration.

B. Source Code – q2_dfa.py

```
"""Q2 - Deterministic Finite Automaton (DFA) Simulator."""


class DFA:
    def __init__(self, states, alphabet, transitions, start, finals):
        self.states, self.alphabet = set(states), set(alphabet)
        self.transitions, self.start, self.finals = transitions, start, set(f
        inals) self._validate()

    def _validate(self):
        """Check that the DFA description is well formed and comple
        te."""if self.start not in self.states:
            raise ValueError("Initial state not in state s
        et")if not self.finals <= self.states:
            raise ValueError("Final state(s) not in state s
        et")for s in self.states:
            for a in self.alphabet:
                if (s, a) not in self.transitions:
                    raise ValueError(f"Missing transition for ({s},
                {a})")if self.transitions[(s, a)] not in self.state
                s:
                    raise ValueError(f"Transition ({s}, {a}) leads to unknown state")

    def simulate(self, string):
        """Return (path, accepted). Path is None if a symbol is not in the alphab
        et."""state, path = self.start, [self.start]
        for ch in string:
            if ch not in self.alphab
                et: return None, Fals
                e
            state = self.transitions[(state, c
            h)]path.append(state)
        return path, state in self.finals


def ends_with_ab_dfa() -> DFA:
    """Built-in DFA for the language of strings ending with '
    ab'."""t = {("q0", "a"): "q1", ("q0", "b"): "q0",
         ("q1", "a"): "q1", ("q1", "b"): "q2",
         ("q2", "a"): "q1", ("q2", "b"): "q0"}
    return DFA({"q0", "q1", "q2"}, {"a", "b"}, t, "q0", {"q2"})


def read_dfa() -> DFA:
    """Accept the DFA description from the user."""
    states = input("Enter states (space separated)    : ").split(
    ) alphabet = input("Enter input alphabet (space separated)     :
    ").split()print("Enter transitions as: <state> <symbol>
    <next state>") transitions = {}
    for _ in range(len(states) * len(alphabe
        t)):s, a, n = input("      " ) . s p
        l i t ( ) transitions[(s, a)] = n
    start = input("Enter initial state             : ").strip(
    )finals = input("Enter final state(s) (space separated)
                                                   : ").split()
    return DFA(states, alphabet, transitions, start, finals)


def test_strings(dfa, string
    s):for w in strings:
        path, ok = dfa.simulate(
        w)print(f"\nInput String
                                 : '{w}'
        ")if path is None:
            print("Transition Path : Invalid symbol in in
        put")else:
            print("Transition Path :", " -> ".join(pa
        th))print("Result :", "Accepted" if ok else "Rejecte
        d")


def main():
    print("1. Use built-in DFA (strings ending with 'ab')\n2. Enter my ow
    n DFA") try:
        dfa = ends_with_ab_dfa() if input("Choice: ").strip() == "1" else rea
    d_dfa()except (ValueError, KeyError) as err:
```

```
        print("Invalid DFA description:",
        err) return
    n = int(input("How many strings to test? "))
    test_strings(dfa, [input(f"String {i + 1}: ").strip() for i in range(n)])


if __name__ == "__main_
    _": main()
```

C. Sample Output

Run 1 – built-in DFA (strings ending with “ab”), valid and invalid input strings:

```
1. Use built-in DFA (strings ending with 'ab')
2. Enter my own DF
A Choice: 1
How many strings to test?
5 String 1: abaab
String 2: ab
String 3: aba
String 4: abc
String 5: bbb

Input String   : 'abaab'
Transition Path : q0 -> q1 -> q2 -> q1 -> q1 ->
q2 Result      : Accepted

Input String   :      ' a b '
Transition Path : q0 -> q1 ->
q2 Result      : Accepted

Input String   : 'aba'
Transition Path : q0 -> q1 -> q2 ->
q1 Result      : Rejected

Input String   : 'abc'
Transition Path : Invalid symbol in in
put Result     : Rejected

Input String   : 'bbb'
Transition Path : q0 -> q0 -> q0 ->
q0 Result      : Rejected
```

Run 2 – user-defined DFA (accepts binary strings containing an odd number of 1s):

```
1. Use built-in DFA (strings ending with 'ab')
2. Enter my own DF
A Choice: 2
Enter states (space separated)      : q0 q1
Enter input alphabet (space separated)     :
0 1
Enter transitions as: <state> <symbol> <next st
  ate> q0 0 q0
  q0 1 q1
  q1 0 q1
  q1 1 q0
Enter initial state                    : q 0
Enter final state(s) (space separated)   :
q1 How many strings to test? 2
String 1: 1101
String 2: 1100

Input String   : '1101'
Transition Path : q0 -> q1 -> q0 -> q0 ->
q1 Result      : Accepted

Input String   : '1100'
Transition Path : q0 -> q1 -> q0 -> q0 ->
q0 Result      : Rejected
```

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input

Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix

Expected Output

Display all matching patterns based on user selection.

ANSWER – Section 3

A. Algorithm / Approach

Search	Regular expression	Explanation
Date	\b(?:0?[1-9]|[12][0-9]|3[01])([/.-])(?:0?[1-9]|1[0-2])\1[0-9]{4}\b	dd/mm/yyyy; day 1–31, month 1–12, same separator (/ . -) used twice through back-reference \1
Phone	(?<![0-9])[6-9][0-9]{9}(?![0-9])	10-digit number starting 6–9, not part of a longer digit string
Hashtag	(?<!\w)#\w+	'#' followed by word characters
Mention	(?<![\w.])@\w+	'@username'; look-behind avoids matching e-mail addresses
Prefix	\bprefix\w*	words beginning with the user's prefix (case-insensitive)
Suffix	\w*suffix\b	words ending with the user's suffix (case-insensitive)
Word	\bword\b	exact whole-word match (case-insensitive)

- The user's prefix/suffix/word goes through re.escape() so special characters cannot break the pattern; empty keys and invalid menu choices are handled with messages.
- The menu loops until Exit. Options 7 (Word Search) and 8 (Exit) come after the six menu items given in the question, so the Word Search requirement of the problem statement is covered while items 1–6 stay unchanged.
- **Complexity:** each search is one linear scan, O(n) time for text length n; O(m) space for m matches.

B. Source Code – q3_search.py

```
"""Q3 - Smart Pattern Matching Engine using Regular Expressio
ns."""
import re

DATE_RE = re.compile(r"\b(?:0?[1-9]|[12][0-9]|3[01])([/.-])(?:0?[1-9]|1[0-2])\1[0-9]{4}\b")
```

```
PHONE_RE = re.compile(r"(?<![0-9])[6-9][0-9]{9}(?![0-9])")
HASHTAG_RE = re.compile(r"(?<!\w)#\
w+") MENTION_RE = re.compile(r"(?<![\
w.])@\w+")

SAMPLE_TEXT = """Meeting on 12/09/
2026 Call 9876543210
#NLP
@OpenAI
natural language processing"""

def search_dates(text): return [m.group() for m in DATE_RE.findite
r(text)] def search_phones(text): return PHONE_RE.findall(text)
def search_hashtags(text): return HASHTAG_RE.findall
(text) def search_mentions(text): return MENTION_RE.f
indall(text)

def search_prefix(text, prefix):
    return re.findall(rf"\b{re.escape(prefix)}\w*", text, re.IGNORECASE)

def search_suffix(text, suffix):
    return re.findall(rf"\w*{re.escape(suffix)}\b", text, re.IGNORECASE)

def search_word(text, word):
    return re.findall(rf"\b{re.escape(word)}\b", text, re.IGNORECASE)

def show(title, result
    s):print(f"\n{tit
    le}") if results:
        for i, r in enumerate(results,
            1):print(f"    {i}. {r}")
    else:
        print(" No match found")

def read_text():
    print("Enter text (type END on a new line to finish; END alone = use sample t
    ext):") lines = []
    while (line := input()) != "EN
        D": lines.append(line)
    return "\n".join(lines) if lines else SAMPLE_TEXT

def main():
    text = read_text()
    print("\n----- TEXT -----\n" + text + "\n----------")
    while True:
        print("\nUser Menu\n1.Search Date\n2.Search Phone Number\n3.Search Hash
              tag\n" "4.Search Mention\n5.Search Prefix\n6.Search Suffix\n7.Search W
              ord\n8.Exit")
        ch = input("Enter choice: ").strip()
        if ch == "1": show("Dates found:", search_dates(text))
        elif ch == "2": show("Phone numbers found:", search_phones(
        text))elif ch == "3": show("Hashtags found:", search_hash
        tags(text)) elif ch == "4": show("Mentions found:", search
        _mentions(text))elif ch in ("5", "6", "7"):
            key = input("Enter the prefix/suffix/word: ").s
            trip() if not key:
                print("Search key cannot be empty"); continue
            fn, name = {"5": (search_prefix, "Prefix"), "6": (search_suffix, "S
                        uffix"), "7": (search_word, "Word")}[ch]
            show(f"{name} '{key}' matches:", fn(text,
        key))elif ch == "8": print("Exiting..."); bre
        ak
        else: print("Invalid choice, try again")


if __name__ == "__main_
    _": main()
```

C. Sample Output

Input text from the question paper, every menu option, a no-match case and an invalid choice:

```
Enter text (type END on a new line to finish; END alone =
use sample text): Meeting on 12/09/2026
Call 987654
3210 #NLP
@OpenAI
natural language proc
```

```
essing END

----- TEXT -----
Meeting on 12/09
/2026 Call 98765
43210
#NLP
@OpenAI
natural language processing
----------------

User Menu
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search W
ord 8. Exit
Enter choice: 1

Dates found:
  1. 12/09/2026

Enter choice

: 2 Phone numb

ers found:
  1. 9876543210

Enter choic

e: 3 Hashtag

s found:
  1. #NLP

Enter choic

e: 4 Mentio

ns found:
  1. @OpenAI

Enter choice: 5
Enter the prefix/suffix/word: lang

Prefix 'lang' matches:
  1. language

Enter choice: 6
Enter the prefix/suffix/word: ing

Suffix 'ing' matches:
  1. Meeting
  2. processing

Enter choice: 7
Enter the prefix/suffix/word: natural

Word 'natural' matches:
  1. natural

Enter choice: 6
Enter the prefix/suffix/word: zzz
```

```
Suffix 'zzz' mat
  ches:
  No match found

Enter choice: 9
Invalid choice, try again

Enter choic
e: 8 Exitin
g...
```